

1. [Exercici 1] (2 punts):

Sigui $X = \{(a_1, b_1), \dots, (a_n, b_n)\}$ un conjunt d' n intervals no buits. Direm que un conjunt de punts $P = \{p_1, \dots, p_k\}$ *intercepta* un conjunt d'intervals X si tot interval a X conté com a mínim un punt de P , és a dir,

$$\forall i : 1 \leq i \leq n : (\exists j : 1 \leq j \leq k : a_i \leq p_j \leq b_i).$$

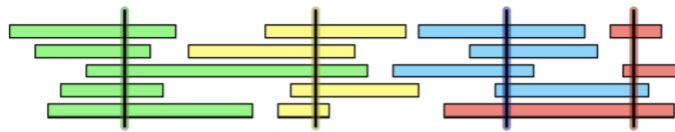


Figura 1: Exemple d'un conjunt d'intervals interceptats per 4 punts (representats com a segments verticals).

- Descriuiu i analitzeu un algorisme eficient que donats el conjunt d'intervals X i el conjunt de punts P , calculi el subconjunt més petit de P que intercepta X , o bé que determini que P **no** intercepta a X .
- Per a la Festa FIB d'aquest any s'han plantejat un conjunt d'activitats per tot el Campus Nord. Cada activitat té una hora i minut d'inici i finalització. Donat que es vol fer una bona difusió a xarxes d'aquell dia, els organitzadors han contractat un servei de fotografia per tal de recollir instantànies de totes les activitats. L'empresa contractada pot enviar fins a n fotògrafs per tal que fotografiïn activitats simultàniament. Donat que és un servei car, es vol minimitzar les vegades que l'equip de fotògrafs ha de desplaçar-se al campus.

Els organitzadors de la Festa FIB s'han posat en contacte amb estudiants d'Algorísmia per tal que els ajudin a decidir en quins moments requerir el servei dels fotògrafs per minimitzar-ne el cost. Descriuiu com resoldríeu i implementaríeu aquest problema de la manera més eficient que se us acudeixi.

Una solució:

(vegeu la solució a l'exercici 3 de l'examen parcial d'aquest quadrimestre)

2. [Exercici 2] (2,5 punts):

Dos jugadors estan jugant al joc dels pots d'or. En aquest joc hi ha n pots d'or disposats en línia, $\{p_1, \dots, p_n\}$, i cada pot p_i conté una determinada quantitat $c_i > 0$ de monedes d'or. Els jugadors juguen de manera alternada i, a cada torn, el jugador que té el torn pot escollir un pot d'un dels extrems de la línia que hi hagi en aquell moment. Per exemple, el jugador que comença el joc pot escollir quedar-se amb el pot p_1 o amb el pot p_n . Si decideix quedar-se amb p_1 , aleshores l'altre jugador podrà escollir entre p_2 i p_n al següent torn; en canvi, si el primer jugador escull p_n , l'altre jugador haurà de triar entre p_1 i p_{n-1} .

El guanyador és el jugador que, una vegada no quedin pots per agafar, tingui una quantitat més gran de monedes.

Dissenyeu un algorisme eficient per calcular quin és el nombre màxim de monedes que podria obtenir en aquest joc el jugador que comença primer, suposant que els dos jugadors volen guanyar i, per tant, ambdós juguen de manera intel·ligent per tal d'obtenir el màxim nombre possible de monedes. Doneu també la seqüència de pots que hauria d'anar escollint a cada torn per aconseguir aquesta quantitat.

Una solució:

La idea és trobar una estratègia òptima que faci que el jugador guanyi, sabent que el seu oponent està jugant de manera òptima (igual que ell). En un instant de joc en què els pots disponibles són $\{p_i, \dots, p_j\}$ (on i i j denoten el primer i l'últim pot de la línia, respectivament), el jugador té dues opcions, que condicionen les dues tries que després podrà fer l'oponent:

- (a) Si el jugador tria el pot p_i , l'oponent haurà de triar entre $\{p_{i+1}, \dots, p_j\}$, i poden passar dues coses:
 - i. Si l'oponent tria el pot p_{i+1} , al següent torn el jugador ha de jugar amb $\{p_{i+2}, \dots, p_j\}$.
 - ii. Si l'oponent tria el pot p_j , al següent torn el jugador ha de jugar amb $\{p_{i+1}, \dots, p_{j-1}\}$.
- (b) Si el jugador tria l'olla p_j , l'oponent haurà de triar entre $\{p_i, \dots, p_{j-1}\}$, i poden passar dues coses:
 - i. Si l'oponent tria el pot p_i , al següent torn el jugador ha de jugar amb $\{p_{i+1}, \dots, p_{j-1}\}$.
 - ii. Si l'oponent tria el pot p_{j-1} , al següent torn el jugador ha de jugar amb $\{p_i, \dots, p_{j-2}\}$.

Com que l'oponent està jugant de manera intel·ligent (és a dir, òptima, per guanyar), intentarà minimitzar els punts del jugador, és a dir, l'oponent farà una elecció que deixarà al jugador amb el mínim de monedes.

Per tant, podem definir recursivament el problema com:

$$OR(i, j) = \begin{cases} c_i & \text{si } i = j \\ \max\{c_i, c_j\} & \text{si } i + 1 = j \text{ (cas redundant)} \\ \max\{c_i + \min\{OR(i+2, j), OR(i+1, j-1)\}, \\ \quad c_j + \min\{OR(i+1, j-1), OR(i, j-2)\}\} & \text{altrament} \end{cases}$$

Implementada com a programa recursiu, la complexitat temporal de la solució anterior és exponencial i ocupa espai a la pila de crides.

Però el problema té *subestructura òptima*, ja que es pot desglossar en subproblemes més petits, que encara es poden desglossar en subproblemes més petits, etc., i una solució òptima a un problema es constitueix a partir de les solucions òptimes dels subproblemes. No pot passar que una solució òptima s'obtingui a partir de solucions dels subproblemes que no són òptimes.

El problema també mostra *subproblemes superposats*, de manera que acabarem resolent el mateix subproblema una vegada i una altra. Sabem que els problemes amb subestructura òptima i subproblemes superposats es poden resoldre mitjançant programació dinàmica, on les solucions de subproblemes es memoritzen en lloc de calcular-les de nou.

Aprofitem aquestes dues característiques per definir una taula T que memoritzarà les solucions als subproblemes, de manera que a $T[i][j]$ guardarem el resultat de $OR(i, j)$. Observant les dependències de la recurrència veiem que, per tal de tenir ja calculades les solucions als subproblemes que es necessitin (i fer, doncs, que consultar-les sigui temps $O(1)$), haurem d'omplir aquesta matriu (o, el que és el mateix, fer els càlculs) de manera iterativa per diagonals (per exemple), començant a la diagonal principal (cas base). Només necessitem emplenar la meitat superior de la matriu. L'últim valor que emplenarem serà $T[1][n]$, que correspon a $OR(1, n)$, que el nostre objectiu (el valor de la recurrència que resol el problema original).

Tenint això en consideració, la complexitat temporal de la solució per programació dinàmica (tant top-down com bottom-up) és $O(n^2)$ (resoldrem $O(n^2)$ subproblemes amb cost $O(1)$) i requereix $O(n^2)$ espai addicional, on n és el nombre total de pots d'or.

Observacions:

- Noteu que, per considerar que l'oponent juga de manera intel·ligent, el cas recursiu han de fer un màxim de **mínims**. Si fem un màxim de màxims, aleshores l'oponent estaria ajudant el jugador a guanyar.
- La recurrència proposada es pot separar en dues recurrències interrelacionades: una que descriu el guany directe del jugador, i una altra que descriu el guany de l'oponent. És, però, innecessari perquè tots dos jugadors són iguals.

Una altra solució:

El problema també es pot formalitzar i resoldre mitjançant la següent forma (més concisa) de la recurrència:

$$OR(i, j) = \begin{cases} c_i & \text{si } i = j \\ \sum_{i \leq k \leq j} c_k - \min \{OR(i+1, j), OR(i, j-1)\} & \text{altrement} \end{cases}$$

on $\sum_{i \leq k \leq j} c_k$ és el total de monedes d'or a la línia $\{p_i, \dots, p_j\}$, i es pot calcular de manera eficient amb memorització. Aleshores, el cost asimptòtic d'aquesta solució és també $O(n^2)$, en temps i espai.

3. [Exercici 3] (2,5 punts):

Una graella $n \times n$ és un graf no dirigit amb n^2 vèrtexs organitzats en n files i n columnes. Denotem el vèrtex a la i -èsima fila i la j -èsima columna per (i, j) . Cada vèrtex (i, j) té exactament quatre veïns $(i-1, j)$, $(i+1, j)$, $(i, j-1)$ i $(i, j+1)$, excepte els vèrtexs a la vora, per als quals $i=1$, $i=n$, $j=1$, o $j=n$. Siguin $(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)$ vèrtexs diferents, anomenats *terminals*, a la graella $n \times n$. El problema d'escapament consisteix a determinar si a la graella hi ha m camins disjunts (en vèrtexs) que connectin els terminals amb qualsevol m vèrtexs diferents de la vora de la graella.

- Doneu i analitzeu un algorisme eficient per a resoldre el problema d'escapament.
- Ara suposem que l'entrada del problema d'escapament inclou també una llista d'arestes prohibides (és a dir, arestes que no poden aparèixer als camins) i una altra llista d'arestes requerides (és a dir, que han d'aparèixer almenys a un camí). Doneu i analitzeu un algorisme eficient per decidir si aquesta versió del problema d'escapament té solució. En cas que en tingui, el vostre algorisme haurà de proporcionar una solució.

Una solución (Apartado a)

Una forma de resolver el “problema d'escapament” es reducirlo al problema de búsqueda de caminos disjuntos. Para ello, vamos a modelar nuestro problema como un grafo $G = (V, E)$ en la que $V = \{s, t\} \cup P$ siendo $P = \{p_{i,j}\}$ los vértices de la graella. Respecto a las aristas E , para cada vértice $p_{i,j}$ existirán 4 aristas con sus vecinos $(p_{i,j}, p_{i+1,j})$, $(p_{i-1,j}, p_{i,j+1})$, $(p_{i,j}, p_{i,j-1})$, $(p_{i,j}, p_{i,j})$ (excepto los vértices del borde). Además, conectaremos todos vértices del borde con s y los terminales con t (Figura 2, panel A).

Para determinar si existen m caminos disjuntos (en vértices) que conectan los terminales con cualquiera de los m vértices diferentes del borde de la graella, utilizaremos el Teorema de Menger por el que el número máximo de caminos disjuntos (en aristas) $s \rightsquigarrow t$ en G es igual al número de aristas mínimo que atraviesa un s - t corte, o lo que es lo mismo, igual al valor del máximo flujo f^* en $\mathcal{N}$. Para que el problema tenga solución, al menos se ha de cumplir que $m \leq 4(n-1)$.

No obstante, antes de proceder a buscar caminos disjuntos (en vértices) en nuestro grafo, tenemos que adaptar nuestro grafo no-dirigido para (a) convertir nuestro G en digrafo y (b) imponer la restricción de que los caminos sean disjuntos en vértices. Para ello, aplicaremos dos transformaciones.

- Para convertir el grafo no-dirigido G a digrafo $G' = (V', E')$, por cada arista (v, u) vamos a crear dos aristas dirigidas (v, u) y (u, v) de capacidad 1 (Figura 2, panel B). Un problema podría ser que el flujo f podría usar simultáneamente las dos aristas (v, u) y (u, v) . Sin embargo, tal y como se vio en teoría, en tal caso siempre podemos eliminar ambas aristas del flujo f . El flujo resultante es válido y tiene el mismo valor. De tal forma, si repetidamente eliminamos esas “dobles aristas” del flujo f , el flujo resultante f' tendrá el mismo valor.
- Para imponer la restricción de buscar caminos vértices disjuntos, vamos a “dividir” cada vértice v de G' en v_i y v_o (Figura 2, panel C). El vértice v_i (in) estará conectado con las aristas que antes entraban en v . El vértice v_o (out) estará conectado con las

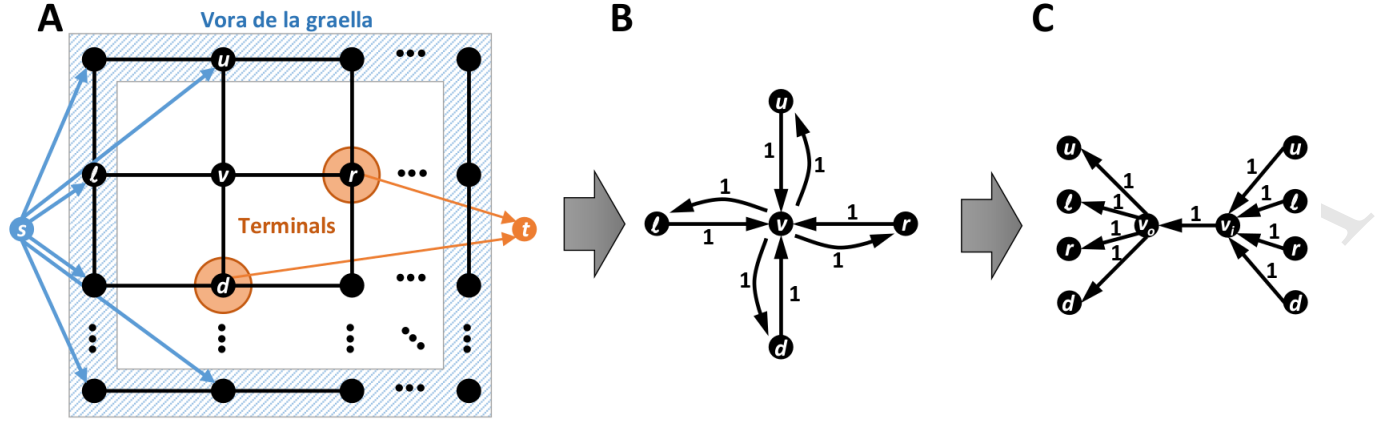


Figura 2: Diagrama de la graella y las transformaciones aplicadas para reducirlo al problema de búsqueda de caminos disjuntos usando un flujo. Notar que en el panel C los vértices $\{u, l, r, d\}$ han sido duplicados para facilitar la visualización.

aristas que antes salían de v . Adicionalmente, crearemos una arista (v_i, v_o) de capacidad 1. Basándonos en el grafo resultante $G'' = (V'', E'')$, forzamos a que cualquier flujo necesariamente solo pueda pasar una vez por (v_i, v_o) (modelando un vértice de la graella).

Formalmente, construimos una red s-t $\mathcal{N} = (V'', E'', s, t, c : \forall e \in E \rightarrow 1)$ en tiempo $O(|V''| + |E''|) = O(n^2)$, lineal en el número de vértices y aristas de la graella. Utilizando el algoritmo de Ford-Fulkerson, podemos computar el flujo máximo f^* en tiempo pseudopolinómico $O(Dn^2)$. Sabemos que el valor del flujo máximo $D = s(f^*)$ no puede exceder la capacidad de los cortes $\langle s, V'' \setminus s \rangle$ y $\langle V'' \setminus t, t \rangle$. Por tanto, $s(f^*) \leq \min\{4(n-1), m\} = m$ y podemos acotar el tiempo polinómico de FF con $O(m \cdot n^2)$.

Una solución (Apartado b)

- Por un lado, para modelar (u, v) aristas prohibidas, solo tenemos que limitar a cero la capacidad de las aristas (u, v) y (v, u) de nuestro digrafo $G' = (V', E')$ (o eliminarlas directamente).
- Por otro lado, para modelar aristas requeridas, podemos modelar el problema como una red con demandas y cotas inferiores. En este caso, las aristas requeridas (u, v) han de tener cota inferior 1. Debido a la transformación a digrafo realizada en el apartado anterior, no podemos aplicar la cota inferior a las aristas (u, v) y (v, u) de forma excluyente. No obstante, siendo R el conjunto de aristas requeridas, podemos plantear buscar una solución en las $2^{|R|}$ redes con demandas posibles. Reducimos el problema a una red con demandas transformando toda aristas requerida $e = (u, v)$, con $c(e) = 1$ y $l(e) = 1$, de la siguiente forma:
 - Eliminamos la arista dado que $c'(e) = c(e) - l(e) = 0$.
 - Actualizamos las demandas en ambos extremos; $d'(u) = d(u) + l(e)$ y $d'(v) = d(v) - l(e)$.

Por construcción de la red, sabemos que $D = \sum_{v \in T} d(v) = -\sum_{v \in S} d(v)$, donde $S = \{v \in V \mid d(v) < 0\}$ son los vértices origen de las aristas requeridas (vértices fuente)

y $T = \{v \in V \mid d(v) > 0\}$ son los vértices destino de las aristas requeridas (vértices sumidero). Es decir, la red $\mathcal{N} = (V, E, c, l, d)$ admite una circulación si y solo si el flujo máximo $s(f^*) = D$. Además, todas las capacidades y demandas en $\mathcal{N}$ son enteras y, por tanto, $\mathcal{N}$ admite una circulación entera para toda arista. Por tanto, el problema puede resolverse en tiempo $O(2^{|R|} \cdot mn^2)$

4. [Exercici 4] (3 punts: 0,75 per qüestió):

- a) Donat un conjunt d' n elements diferents sense ordenar, podem dissenyar un algorisme que, donat $k < n/4$, doni com a sortida la llista ordenada dels $2k$ elements immediatament més grans que la mediana, en temps $O(n + k \lg k)$? Si la resposta és afirmativa, doneu l'algorisme.

Una solució:

Sí, es posible. El algoritmo que propongo es:

- Guardar los valores en un vector A , con coste $O(n)$.
- Utilizando el algoritmo de selección (QuickSelect) obtener la mediana de A y extraer los elementos que son mayores que la mediana en un vector B , con coste $O(n)$.
- Con QuickSelect obtener el elemento en la posición $2k$ de B y extraer los elementos que son menores o iguales que él en un vector C . De nuevo con coste $O(n)$.
- Ordenar C usando merge sort. C tiene $2k$ elementos así el coste es $O(2k \log 2k) = O(k \log k)$

C contiene los $2k$ elementos posteriores a la mediana y el coste total del algoritmo es $O(n + k \log k)$.

- b) Donat un graf G i un arbre d'expansió mínim T d'aquest graf, suposeu que fem decreïxer el pes d'una de les arestes a T . Demostreu que T continua sent un arbre d'expansió mínim de G .

Una solució:

Como T es un árbol de expansión de G si eliminamos una arista cualquiera $e = (u, v) \in E(T)$ obtenemos un corte (C_u, C_v) en G , tal que $u \in C_u$ y $v \in C_v$, correspondiente a los vértices en los dos subárboles resultantes. Por la regla azul sabemos que e es una arista de peso mínimo en este corte.

Supongamos que $e = (u, v) \in E(T)$ es la arista que rebaja su peso. Como bajamos el peso, e es ahora la arista de menor peso en el corte (C_u, C_v) . Consideremos cualquier otra arista $e' = (u', v') \in T$ tal que $e' \neq e$. Si eliminamos e' , la arista e aparece en uno de los dos subárboles. Por lo tanto, ninguna de las aristas en el corte $(C_{u'}, C_{v'})$ ha cambiado su peso y e' continua siendo una arista de peso mínimo en el corte correspondiente. Como cada una de las aristas de T es la de peso mínimo en algún corte, por la regla azul, T es un MST con la nueva ponderación.

Una altra solució:

Sea $e = (u, v) \in E(T)$ la arista que rebaja su peso y se $a > 0$ la cantidad en que se rebaja este peso. Llamemos w a la asignación inicial de pesos a las aristas y w' a la nueva asignación en la que $w'(e) = w(e) - a$, y el resto de aristas mantiene su peso, si $e' \neq e$, $w'(e) = w(e)$.

Con esta notación $w'(T) = w(T) - a$. Vamos a ver que T es un MST para (G, w') . Sea T' un árbol de expansión de G . Tenemos dos posibles casos

- $e \in E(T')$, entonces $w'(T') = w(T') - a \geq w(T) - a = w'(T)$
- $e \notin E(T')$, entonces $w'(T') = w(T) \geq w(T) \geq w(T) - a = w'(T)$

En los dos casos concluimos que $w'(T) \leq w'(T')$ por lo que continúa siendo un MST en la nueva ponderación.

- c) Diem que un graf $G = (V, E)$ és *petit* si és connex i $|E| = |V|$. Sigui donat un graf petit $G = (V, E)$ juntament amb una assignació de pesos $w : E \rightarrow \mathbb{R}$ i un vèrtex $s \in V$. Es pot determinar en temps $O(|V|)$ si es pot definir la distància des de s a qualsevol altre vèrtex de V i, en cas que ho sigui, computar-la?

Una solució:

El grafo es no dirigido, conexo y tiene $n = |V|$ aristas. Por lo tanto, es un grafo con un único ciclo, y eliminado una arista del ciclo obtenemos un árbol de expansión.

Dada la estructura del grafo podemos interpretar qué es un camino de dos formas diferentes:

- Un camino es un camino en el grafo dirigido que obtenemos al reemplazar cada arista por dos arcos uno en cada dirección. En este caso, si alguno de los pesos es negativo, tendríamos un ciclo de longitud dos con peso negativo, y no se podrían definir algunas de las distancias de s al resto de vértices.
- Un camino es una secuencia de aristas y no permitimos ciclos de longitud dos. En este caso las distancias no se pueden definir cuando el único ciclo del grafo tiene peso negativo. El ciclo se puede obtener utilizando un BFS o un DFS desde s .

Bajo cualquiera de las dos interpretaciones se puede determinar si se puede definir distancias en tiempo $O(|V| + |E|) = O(|V|)$.

Suponiendo que las distancias se pueden definir, necesitamos un algoritmo específico ya que ninguno de los vistos en clase nos daría la cota de tiempo pedida. El algoritmo que propongo es el siguiente:

- Con un BFS desde s obtener el único ciclo $C = v_0, \dots, v_k, v_0$ en G . Siendo v_0 el primer vértice de C al que accede el BFS.
- Eliminamos la arista v_0, v_1 , así obtenemos un árbol de expansión T_0 . Calculamos la distancia de s a cada uno de los vértices de G en el único camino en T_0 .
- Eliminamos la arista v_k, v_0 , así obtenemos un árbol de expansión T_1 . Calculamos la distancia de s a cada uno de los vértices de G en el único camino en T_1 .
- Nos quedamos con la menor de las dos distancias.

Para $u \in V$, el algoritmo examina el único o los dos caminos de s a u y se queda con el que tiene menor peso. Por lo que es correcto. Además, el coste de cada uno de los 4 pasos es $O(|V|)$.

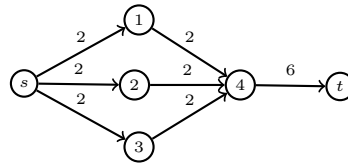
Comentario Un error común es asumir que un BFS en un grafo proporciona la distancia mínima. Un BFS nos permite obtener caminos con longitud mínima pero estos no tienen porque tener distancia mínima.

d) Digueu si és certa o falsa la següent afirmació, i justifiqueu-ne la resposta.

Sigui $\mathcal{N} = (V, E, c, s, t)$ una xarxa de flux. Ens donen un $s - t$ tall (S, T) , i sigui $E(S, T) = \{(u, v) \in E \mid u \in S, v \in T\}$ amb capacitat total α . A més sabem que el valor d'un flux amb valor màxim a $\mathcal{N}$ és α . Sigui $\mathcal{N}'$ la xarxa de flux obtinguda afegint $+1$ a la capacitat de cada aresta a $E(S, T)$. Llavors, el valor d'un flux amb valor màxim a $\mathcal{N}'$ és 2α .

Una solució:

La afirmación es falsa. La siguiente red proporciona un contraejemplo.



Tomando $S = \{s, 1, 2, 3\}$ y $T = \{4, t\}$, $E(S, T)$ tiene capacidad 6. Saturando todas las aristas con flujo igual a su capacidad, tenemos un flujo con valor 6. S y T cumplen las condiciones requeridas. Sin embargo si aumentamos la capacidad de las tres aristas de $E(S, T)$, el flujo no se puede incrementar ya que es imposible que salgan más de 6 unidades de flujo de s (o que lleguen a t).